

Universidad Nacional Autónoma de México

Facultad de Ciencias

Ciencias de la Computación

Práctica 3: Analizador sintáctico descendente recursivo

Compiladores

Semestre 2027-1

Equipo: DFGR

Integrantes: Flores Galeana Daniela - 321162342
Rivera Machuca Gabriel Eduardo - 321057608

Profesora: L. en C.C. Yessica Janeth Pablo Martínez

Ayudante: Pedro Yamil Salazar González

Fecha de entrega: 21 de septiembre de 2026

Índice

1. Objetivo	2
2. Evolución respecto de la Práctica 2	2
2.1. Módulos modificados y agregados	2
2.2. Interfaz incremental incorporada al lexer	2
2.3. Eliminación de la impresión de tokens	2
2.4. Refactorizaciones y funcionalidades conservadas	2
3. Arquitectura e interfaz	3
3.1. Organización del módulo <code>parser</code>	3
3.2. Responsabilidades y flujo	3
3.3. Administración de memoria y manejo de errores léxicos	3
4. Descenso recursivo y gramática	3
5. Expresiones	4
6. Diagnósticos y recuperación	4
7. Resultados y pruebas	5
8. Problemas encontrados	5
9. División del trabajo	5
10. Uso de herramientas de Inteligencia Artificial	5
11. Conclusiones	6
12. Referencias	6

1. Objetivo

El objetivo principal de esta tercera práctica fue implementar un analizador sintáctico descendente recursivo para el lenguaje MiniC. Este componente recibe la secuencia de fichas (tokens) producida por el analizador léxico y determina si un programa se encuentra organizado de acuerdo con las reglas de la gramática libre de contexto establecida. Su papel principal en esta etapa es actuar como un filtro de validación estructural entre el lexer y la futura construcción del Árbol de Sintaxis Abstracta (AST) que se abordará en la siguiente práctica, respondiendo únicamente a la pregunta: ¿El programa está sintácticamente bien escrito?

2. Evolución respecto de la Práctica 2

2.1. Módulos modificados y agregados

Se mantuvo la modularidad del compilador incorporando un nuevo módulo dedicado al análisis sintáctico (`src/parser/parser.c` y `include/parser/parser.h`), el cual interactúa estrechamente con los módulos existentes del lexer y los tokens.

2.2. Interfaz incremental incorporada al lexer

Se transformó el analizador léxico de un escaneo masivo con impresión directa a una interfaz incremental bajo demanda mediante la función `lexer_next_token(Lexer *lexer, Token *out)`. Esto permite que el parser solicite fichas de manera individual conforme las necesita para su análisis descendente.

2.3. Eliminación de la impresión de tokens

Se eliminó por completo la impresión en consola de los tokens individuales generada por el lexer durante la ejecución normal. La salida estándar (`stdout`) quedó reservada exclusivamente para el mensaje global de éxito o los errores de uso, cumpliendo estrictamente con el flujo del compilador.

2.4. Refactorizaciones y funcionalidades conservadas

Se refactorizó la inicialización del lexer (`lexer_init`) para unificar los códigos de retorno estándar de éxito y error. Asimismo, se conservaron intactas todas las funcionalidades logradas en la Práctica 2: la gestión de memoria dinámica para lexemas, el seguimiento preciso de líneas y columnas, el reconocimiento de palabras reservadas, literales, operadores compuestos y el manejo de comentarios y errores léxicos.

3. Arquitectura e interfaz

3.1. Organización del módulo parser

El módulo se encarga de gestionar el estado del análisis sintáctico mediante la estructura `Parser`, la cual almacena un puntero al analizador léxico activo, el token actual de anticipación (`current`), el token previamente consumido (`previus`), y banderas de control de errores (`had_error` y `panic_mode`).

3.2. Responsabilidades y flujo

- **Lexer:** Extrae caracteres del archivo fuente y los agrupa en tokens incrementales.
- **Parser:** Consume dichos tokens aplicando las reglas gramaticales mediante descenso recursivo, detecta desviaciones y coordina la recuperación ante errores.
- **main.c:** Coordina la apertura del archivo, la inicialización escalonada del lexer y del parser, ejecuta la validación global y libera los recursos del sistema devolviendo el código de salida adecuado.

3.3. Administración de memoria y manejo de errores léxicos

La propiedad de cada token y su respectivo lexema dinámico es gestionada de forma estricta: el parser destruye el token actual e histórico mediante `token_destroy` al avanzar en la lectura. Si el lexer detecta un carácter inválido y emite un token de tipo `ERROR`, el parser lo intercepta en la función de avance, reporta el diagnóstico léxico a `stderr` de manera única y continúa sin interrumpir abruptamente el análisis sintáctico.

4. Descenso recursivo y gramática

El parser fue implementado utilizando la técnica de descenso recursivo, donde cada símbolo no terminal de la gramática EBNF de MiniC cuenta con una función homónima. Mediante el uso de un token de anticipación (*lookahead*), el método `parse_statement` inspecciona el tipo de la ficha actual y selecciona la producción adecuada mediante una estructura condicional:

- **Declaraciones** (`parse_declaration`): Reconocen tipos (`int` o `bool`), identificadores, una asignación opcional y finalizan obligatoriamente con punto y coma (;).
- **Asignaciones** (`parse_assignment`): Procesan el identificador, el operador `=`, una expresión y el punto y coma final.
- **Impresión** (`parse_print`): Validan la palabra clave `print`, paréntesis, una expresión interna y el cierre con ;.

- **Condicionales y Ciclos (`parse_if` / `parse_while`):** Reconocen la estructura de control, la condición booleana entre paréntesis y la sentencia asociada, manejando opcionalmente la rama `else` del condicional más cercano.
- **Bloques (`parse_block`):** Agrupan múltiples sentencias delimitadas por llaves (`{` y `}`).

5. Expresiones

Para respetar rigurosamente la precedencia y asociatividad de los operadores descritas en el lenguaje MiniC, la gramática de expresiones se estructuró en una jerarquía estricta de funciones donde cada nivel delega en el operador de mayor prioridad:

1. `parse_expression()` → llamadas a `parse_or()` (`||`)
2. `parse_and()` (`&&`)
3. `parse_equal()` (`==`, `!=`)
4. `parse_comparison()` (`<`, `<=`, `>`, `>=`)
5. `parse_add()` (`+`, `-`)
6. `parse_multiplicative()` (`*`, `/`)
7. `parse_unary()` (Operador unario negativo - con asociatividad a la derecha mediante recursividad)
8. `parse_primary()` (Literales enteros, booleanos, identificadores o expresiones agrupadas entre paréntesis).

6. Diagnósticos y recuperación

Cuando el token actual no coincide con lo esperado por la gramática, se invoca a `syntax_error`, imprimiendo un mensaje estricto en `stderr` con el formato:

```
Error sintactico [linea:columna]: se esperaba <elemento>, pero se
encontro '<lexema>'.
```

Para evitar la parálisis total del compilador ante el primer fallo, se implementó una estrategia de sincronización en modo pánico (*panic mode*) mediante la función `synchronize()`. Al activarse un error, el parser descarta tokens de manera controlada hasta encontrar un punto y coma (`;`) o palabras reservadas que inician una nueva sentencia (`int`, `bool`, `if`, `while`, `print`, `,`, llaves), garantizando que no existan ciclos infinitos y permitiendo detectar múltiples errores independientes a lo largo del archivo. Al finalizar el análisis de las sentencias, se verifica obligatoriamente la presencia de `TOKEN_EOF` para asegurar el consumo total del archivo fuente.

7. Resultados y pruebas

El sistema fue evaluado utilizando la batería completa de pruebas públicas proporcionadas para la Práctica 3, las cuales validan tanto programas correctos (p01 a p08) como casos inválidos con errores sintácticos, léxicos y múltiples fallos simultáneos (p09 a p17). El ejecutable `minic` responde con los códigos de salida apropiados (0 para éxito y 1 para errores detectados) y emite los diagnósticos exclusivamente por el flujo de errores estándar. Además se implementaron pruebas privadas en la carpeta `tests/equipo/inputs` en las cuales se prueban casos como códigos sintácticamente correctos y otros casos con errores o con símbolos inexistentes en la gramática como `@` lo cual genera errores los cuales son los esperados; sin embargo se encontró un problema con una secuencia `if, else` la cuál se describe en la siguiente sección.

8. Problemas encontrados

Durante el desarrollo de la sincronización en modo pánico, experimentamos bloqueos y bucles infinitos en pruebas con contenido sobrante o errores múltiples debido a retornos prematuros evaluados con el token previo. Tras analizar el flujo de avance del lookahead, rediseñamos la función `synchronize` para asegurar el consumo seguro de los puntos y coma y la detención oportuna en las fronteras de las sentencias. Se encontró un error con `if else` al hacer las pruebas, el error consta en que si se recibe por ejemplo `if(x==0) print(1) else print(2)` aunque se debería leer como correcto el parser no lo entiende como que el `else` pertenece al `if`, si no que toma el `else` como un elemento aislado que le falta un `if` y queda en un bucle infinito, se detectó el error, sin embargo se nos acabó el tiempo para corregirlo.

9. División del trabajo

El desarrollo de la práctica se realizó de manera colaborativa:

- **Gabriel Eduardo Rivera Machuca:** Diseño de la interfaz incremental del lexer, implementación de la jerarquía de expresiones con precedencia en el parser y adaptación del flujo principal en `main.c`.
- **Daniela Flores Galeana:** Implementación de las funciones de sentencias de control (`if`, `while`, bloques), pruebas unitarias locales y estructuración del mecanismo de recuperación ante errores sintácticos.

Ambos integrantes revisaron conjuntamente la arquitectura del código y comprenden la totalidad de la implementación.

10. Uso de herramientas de Inteligencia Artificial

Para esta práctica se utilizó Gemini (Google) como herramienta de apoyo técnico.

- **Propósito:** Consultar la teoría de diseño de parsers descendentes recursivos, la estructura del mecanismo de sincronización en modo pánico y la sintaxis avanzada de apuntadores en C.
- **Consultas realizadas:** Dudas sobre la gestión del ciclo iterativo en `parser_advance` para ignorar tokens léxicos de error sin propagar fallos sintácticos redundantes.
- **Apoyo en el reporte:** Organización de secciones y redacción formal orientada a los criterios de la rúbrica institucional.
- **Revisión y comprobación:** Todo el código generado o consultado fue probado, depurado e integrado de manera manual, verificando su éxito con el script de pruebas.

11. Conclusiones

La implementación del analizador sintáctico permitió comprender la importancia de transformar un flujo lineal de fichas en una estructura jerárquica coherente. El dominio del descenso recursivo y el manejo adecuado del token de anticipación sientan las bases conceptuales necesarias para la siguiente etapa del compilador, donde estas mismas funciones de reconocimiento serán extendidas para construir de manera limpia y eficiente los nodos del Árbol de Sintaxis Abstracta (AST).

12. Referencias

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2da ed.). Pearson.
- Fischer, C. N., Cytron, R. K., & LeBlanc, R. J. (2009). *Crafting a compiler*. Pearson.
- GeeksforGeeks. (s.f.). *Dynamic memory allocation in C using malloc(), calloc(), free() and realloc()*. <https://www.geeksforgeeks.org/dynamic-memory-allocation-in-c-using-malloc-calloc-free-and-realloc/>.
- GeeksforGeeks. (s.f.). *Introduction of lexical analysis*. <https://www.geeksforgeeks.org/compiler-design/introduction-of-lexical-analysis/>
- International Organization for Standardization. (2018). *Information technology — Programming languages — C* (ISO/IEC 9899:2018).
- Kernighan, B. W., & Ritchie, D. M. (1988). *The C programming language* (2da ed.). Prentice Hall.
- Pablo Martínez, Y. J. (2026). *Compiladores_Practica2.pdf y GUIA_INTEGRACION_P2.md* [Material de clase]. Facultad de Ciencias, Universidad Nacional Autónoma de México.

- Valgrind Developers. (s.f.). *Valgrind user manual*. <https://valgrind.org/docs/manual/manual.html>